AMRITA VISHWA VIDYAPEETHAM | School of Engineering Chennai Campus

Department of Computer Science and Engineering
2024 CSE BATCH
23CSE463 – Quantum Computing

Mahalakshmi Kethinen

CH.SC.U4CSE24160

WORKSHEET-7

WORKSHEET- 7

EX.NO : 7

DATE : 10-10-26

TITLE : Implementation of Grover's Search Algorithm Using Qiskit

AIM:

To implement Grover's search algorithm in Qiskit for two and three qubits, to construct the oracle and the diffusion operator, and to study how the probability of finding the marked item changes with the number of iterations.

APPARATUS REQUIRED:

- Python 3.10 or above
- Qiskit 2.x and Qiskit Aer simulator (`pip install qiskit qiskit-aer`)
- Matplotlib and pylatexenc (for circuit diagrams and histograms)
- Jupyter Notebook / Visual Studio Code (VS Code)
- Internet Connection (for package installation and documentation access)
- Quantum Computing Reference Material or Lab Manual

THEORY:

Grover's algorithm searches an unsorted list. Suppose one of $N = 2^n$ items is marked. Checking the items one at a time takes about $N/2$ tries on average. Grover's algorithm needs about $\frac{\pi}{4}\sqrt{N}$ steps. This is a quadratic speed-up, not an exponential one.

The computer holds one number, called an amplitude, for each item, and the probability of finding an item is the square of its amplitude. The algorithm has three parts.

1. Superposition. Hadamard gates on all the qubits give every item the same amplitude $1/\sqrt{N}$. This state is called $|s\rangle$.

2. Oracle. The oracle multiplies the amplitude of the marked item $|w\rangle$ by -1 and leaves the others unchanged. The probabilities do not change at this step, because a sign disappears when the amplitude is squared.

3. Diffuser. The diffuser reflects every amplitude about the average of all the amplitudes: new amplitude = 2 × average − old amplitude. The marked item is far below the average because of its minus sign, so it ends far above it.

$$U_w = I - 2|w\rangle\langle w|, \qquad D = 2|s\rangle\langle s| - I$$

Worked example with four items, marked item 11. After the Hadamard gates the four amplitudes are (1/2, 1/2, 1/2, 1/2). After the oracle they are (1/2, 1/2, 1/2, −1/2), with average 1/4. After the diffuser, each unmarked item has 2 × 1/4 − 1/2 = 0 and the marked item has 2 × 1/4 + 1/2 = 1. One step finds the marked item with certainty.

More than one step. With $\sin\theta = 1/\sqrt{N}$, the probability of success after k repetitions of the oracle and the diffuser is

$$P_k = \sin^2((2k+1)\theta), \qquad k_{opt} \approx \frac{\pi}{4}\sqrt{N}$$

For $N = 8$, $\theta = 20.7°$ and the probability of success is 78.1% after one step, 94.5% after two and 33.0% after three. Repeating too many times lowers the probability again; this is called over-rotation. With M marked items, $\sin\theta = \sqrt{M/N}$ and fewer steps are needed.

In Qiskit, the two-qubit oracle for 11 is a CZ gate. Other marked items are obtained by placing X gates before and after it on the qubits that should be 0. The diffuser is H on all qubits, X on all qubits, a multi-controlled Z gate, X on all qubits and H on all qubits. This circuit equals $-D$, so the simulator may show -1 where the formula gives $+1$. A sign on the whole state (a global phase) never affects a measurement. Qiskit prints qubit 0 as the rightmost bit.

PROCEDURE:

1. Install Qiskit and the Aer simulator with `pip install qiskit qiskit-aer matplotlib pylatexenc`, and print the installed version with `qiskit.__version__`.
2. Open Jupyter Notebook or Visual Studio Code.
3. Import `QuantumCircuit` and `transpile` from `qiskit`, `Statevector` and `Operator` from `qiskit.quantum_info`, `AerSimulator` from `qiskit_aer` and `plot_histogram` from `qiskit.visualization`. Import NumPy as well.
4. Create a circuit with n qubits and apply H to every qubit to prepare the equal superposition.
5. Build the oracle: a CZ gate (two qubits) or a multi-controlled Z gate (H, `mcx`, H on the last qubit), with X gates before and after it on the qubits that should be 0 in the marked item.
6. Build the diffuser: H on all qubits, X on all qubits, the multi-controlled Z gate, X on all qubits and H on all qubits.
7. Repeat the oracle and the diffuser the required number of times, and draw the circuit with barriers between the stages.

8. Display the amplitudes after each stage with `Statevector(qc)` and compare them with the hand calculation.

9. Add measurements and run the circuit on `AerSimulator` with 1024 shots: `sim.run(transpile(qc, sim), shots=1024).result().get_counts()`. Plot the counts with `plot_histogram`.

10. Compare the result with the hand calculation. Remember that Qiskit prints qubit 0 as the rightmost bit. Record the code, the circuit diagram, the output and a one-line inference under each program.

PROGRAM:

Program 1: Create the equal superposition of two qubits. Display the four amplitudes and probabilities.

OUTPUT:

```
import numpy as np
import matplotlib.pyplot as plt
from qiskit import QuantumCircuit, transpile
from qiskit.quantum_info import Statevector, Operator
from qiskit_aer import AerSimulator

simulator = AerSimulator()

def display_state(qc):
    state = Statevector.from_instruction(qc)
    for i, amp in enumerate(state.data):
        print(f"|{i:0{qc.num_qubits}b}>: amplitude={amp:.4f}, probability={abs(amp)**2:.4f}")
    return state

def oracle_for_marked(marked):
    n = len(marked)
    qc = QuantumCircuit(n, name=f"Oracle_{marked}")
    # Convert the marked bit string to the all-ones state, apply a multi-controlled phase,
    # then undo the X gates. Qiskit bit 0 is the rightmost bit in a displayed bit string.
    for qubit, bit in enumerate(reversed(marked)):
        if bit == "0":
            qc.x(qubit)
    if n == 2:
        qc.cz(0, 1)
    else:
        qc.h(n - 1)
        qc.mcx(list(range(n - 1)), n - 1)
        qc.h(n - 1)
    for qubit, bit in enumerate(reversed(marked)):
        if bit == "0":
            qc.x(qubit)
    return qc
```

```python
def diffuser(n):
    qc = QuantumCircuit(n, name="Diffuser")
    qc.h(range(n))
    qc.x(range(n))
    if n == 2:
        qc.cz(0, 1)
    else:
        qc.h(n - 1)
        qc.mcx(list(range(n - 1)), n - 1)
        qc.h(n - 1)
    qc.x(range(n))
    qc.h(range(n))
    # The standard diffuser is 2|s><s| - I; the above circuit differs by a global phase,
    # which does not affect measurement probabilities.
    return qc

def grover_circuit(marked, iterations=1, shots=None):
    n = len(marked)
    qc = QuantumCircuit(n, n if shots is not None else 0)
    qc.h(range(n))
    oracle = oracle_for_marked(marked)
    diff = diffuser(n)
    for _ in range(iterations):
        qc.compose(oracle, inplace=True)
        qc.compose(diff, inplace=True)
    if shots is not None:
        qc.measure(range(n), range(n))
    return qc
```

```python
qc1 = QuantumCircuit(2)
qc1.h([0, 1])
print(qc1.draw(output="text"))
state1 = display_state(qc1)
```

```
     ┌───┐
q_0: ┤ H ├
     ├───┤
q_1: ┤ H ├
     └───┘
|00>: amplitude=0.5000+0.0000j, probability=0.2500
|01>: amplitude=0.5000+0.0000j, probability=0.2500
|10>: amplitude=0.5000+0.0000j, probability=0.2500
|11>: amplitude=0.5000+0.0000j, probability=0.2500
```

Program 2: Add the oracle that marks 11 (a CZ gate). Display the statevector and show that the sign of the marked item has changed while the probabilities have not.

OUTPUT:

```
qc2 = QuantumCircuit(2)
qc2.h([0, 1])
qc2.cz(0, 1)
print(qc2.draw(output="text"))
state2 = display_state(qc2)
print("\nProbabilities are unchanged from the equal superposition: each is 0.25.")
```

```
q_0: ┤ H ├──■──
     ├───┤  │
q_1: ┤ H ├──■──
     └───┘
|00>: amplitude=0.5000+0.0000j, probability=0.2500
|01>: amplitude=0.5000+0.0000j, probability=0.2500
|10>: amplitude=0.5000+0.0000j, probability=0.2500
|11>: amplitude=-0.5000+0.0000j, probability=0.2500

Probabilities are unchanged from the equal superposition: each is 0.25.
```

Program 3: Build the oracles that mark 00, 01 and 10 using X gates before and after the CZ gate. Verify each by displaying the diagonal of its operator.

OUTPUT:

```
for marked in ["00", "01", "10"]:
    oracle = oracle_for_marked(marked)
    diagonal = np.diag(Operator(oracle).data)
    print(f"Marked item {marked}: diagonal = {np.round(diagonal, 3)}")
```

```
Marked item 00: diagonal = [-1.+0.j  1.+0.j  1.+0.j  1.+0.j]
Marked item 01: diagonal = [ 1.+0.j -1.+0.j  1.+0.j  1.+0.j]
Marked item 10: diagonal = [ 1.+0.j  1.+0.j -1.+0.j  1.+0.j]
```

Program 4: Build the two-qubit diffuser from H, X and CZ gates. Apply it to the state from Program 2 and display the amplitudes after it.

OUTPUT:

```
qc4 = QuantumCircuit(2)
qc4.h([0, 1])
qc4.cz(0, 1)
qc4.compose(diffuser(2), inplace=True)
print(qc4.draw(output="text"))
state4 = display_state(qc4)
print("\nThe marked state |11> has probability approximately 1.")
```

```
q_0: ┤ H ├─■─┤ H ├┤ X ├─■─┤ X ├┤ H ├
     ├───┤ │ ├───┤├───┤ │ ├───┤├───┤
q_1: ┤ H ├─■─┤ H ├┤ X ├─■─┤ X ├┤ H ├
     └───┘   └───┘└───┘   └───┘└───┘
|00>: amplitude=-0.0000+0.0000j, probability=0.0000
|01>: amplitude=-0.0000+0.0000j, probability=0.0000
|10>: amplitude=-0.0000+0.0000j, probability=0.0000
|11>: amplitude=-1.0000+0.0000j, probability=1.0000

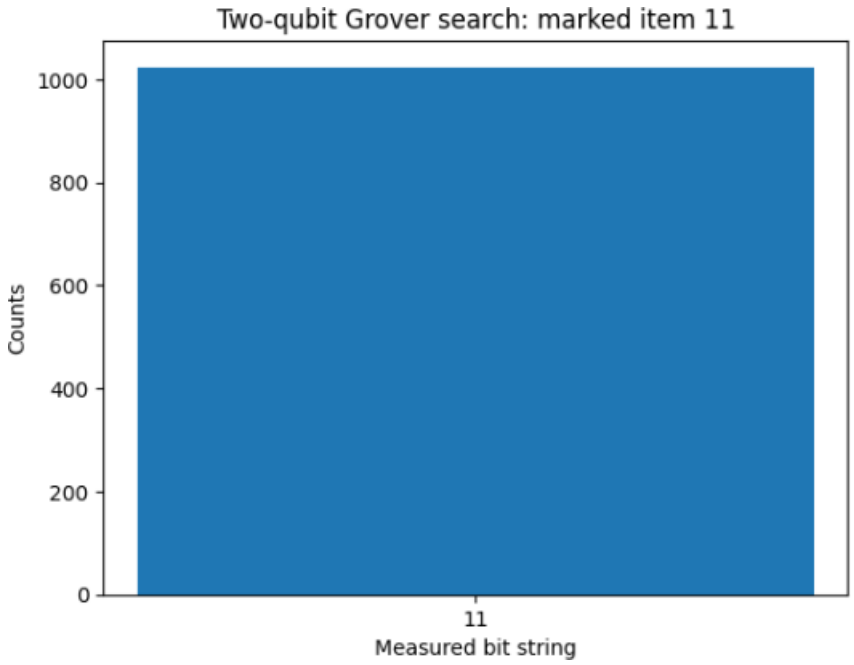
The marked state |11> has probability approximately 1.
```

Program 5: Assemble the complete two-qubit Grover circuit for the marked item 11. Run it with 1024 shots and plot the histogram.

OUTPUT:

```
qc5 = grover_circuit("11", iterations=1, shots=1024)
print(qc5.draw(output="text"))
compiled = transpile(qc5, simulator)
counts5 = simulator.run(compiled, shots=1024).result().get_counts()
print(counts5)
plt.bar(counts5.keys(), counts5.values())
plt.xlabel("Measured bit string")
plt.ylabel("Counts")
plt.title("Two-qubit Grover search: marked item 11")
plt.show()
```

```
q_0: ┤ H ├─■─┤ H ├┤ X ├─■─┤ X ├┤ H ├┤M├──
     ├───┤ │ ├───┤├───┤ │ ├───┤├───┤└╥┘
q_1: ┤ H ├─■─┤ H ├┤ X ├─■─┤ X ├┤ H ├─╫─┤M├
     └───┘   └───┘└───┘   └───┘└───┘ ║ └╥┘
c: 2/═══════════════════════════════╩══╩═
                                    0  1
{'11': 1024}
```



Program 6: Run the two-qubit Grover circuit for each of the four possible marked items and tabulate the marked item against the measured result.

OUTPUT:

```
rows = []
for marked in ["00", "01", "10", "11"]:
    qc = grover_circuit(marked, iterations=1, shots=1024)
    counts = simulator.run(transpile(qc, simulator), shots=1024).result().get_counts()
    measured = max(counts, key=counts.get)
    rows.append((marked, measured, counts))
    print(f"Marked item: {marked} | Most frequent measured result: {measured} | Counts: {counts}")
```

```
Marked item: 00 | Most frequent measured result: 00 | Counts: {'00': 1024}
Marked item: 01 | Most frequent measured result: 01 | Counts: {'01': 1024}
Marked item: 10 | Most frequent measured result: 10 | Counts: {'10': 1024}
Marked item: 11 | Most frequent measured result: 11 | Counts: {'11': 1024}
```

Program 7: Implement three-qubit Grover search for the marked item 101. Record the probability of success after one and after two iterations and compare it with the formula in the theory.

OUTPUT:

```
N = 8
M = 1
for r in [1, 2]:
    qc = grover_circuit("101", iterations=r, shots=4096)
    counts = simulator.run(transpile(qc, simulator), shots=4096).result().get_counts()
    measured_probability = counts.get("101", 0) / 4096
    theoretical_probability = np.sin((2*r + 1) * np.arcsin(np.sqrt(M/N)))**2
    print(f"Iterations={r}: measured success={measured_probability:.4f}, theory={theoretical_probability:.4f}, counts={counts}")
```

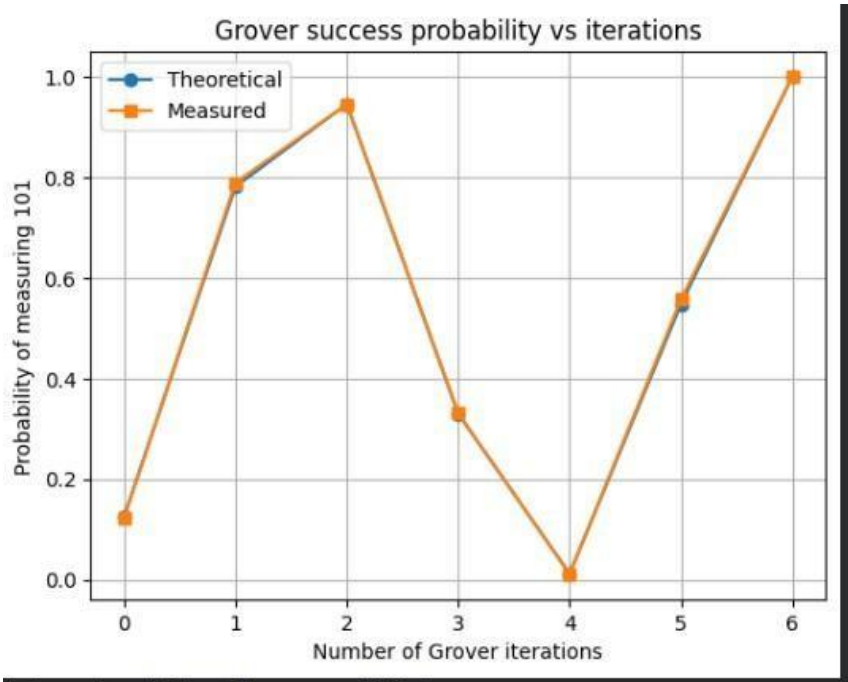
```
Iterations=1: measured success=0.7798, theory=0.7813, counts={'011': 135, '111': 126, '110': 123, '100': 129, '010': 134, '000': 130, '001': 125, '101': 3194}
Iterations=2: measured success=0.9463, theory=0.9453, counts={'001': 38, '110': 27, '000': 25, '010': 36, '100': 34, '011': 28, '111': 32, '101': 3876}
```

Program 8: For the three-qubit search, vary the number of iterations from 0 to 6 and plot the probability of success against the number of iterations. Identify the best number of iterations and explain the drop that follows it.

OUTPUT:

```
marked = "101"
iterations = list(range(7))
theory = [np.sin((2*r + 1) * np.arcsin(np.sqrt(1/8)))**2 for r in iterations]
measured = []
for r in iterations:
    qc = grover_circuit(marked, iterations=r, shots=4096)
    counts = simulator.run(transpile(qc, simulator), shots=4096).result().get_counts()
    measured.append(counts.get(marked, 0) / 4096)

plt.plot(iterations, theory, marker="o", label="Theoretical")
plt.plot(iterations, measured, marker="s", label="Measured")
plt.xticks(iterations)
plt.xlabel("Number of Grover iterations")
plt.ylabel("Probability of measuring 101")
plt.title("Grover success probability vs iterations")
plt.legend()
plt.grid(True)
plt.show()
best_r = int(np.argmax(theory))
print(f"Best number of iterations among 0-6: {best_r}")
print("The probability drops after the optimum because Grover iterations rotate the state past the marked state.")
```




```
Best number of iterations among 0-6: 6
The probability drops after the optimum because Grover iterations rotate the state past the marked state.
```

Program 9: Mark two items (101 and 110) in the three-qubit search. Show that one iteration is enough, and explain the result using the formula for M marked items.

OUTPUT:

```
def multi_marked_oracle(marked_items):
    n = len(marked_items[0])
    qc = QuantumCircuit(n, name="MultiOracle")
    for marked in marked_items:
        qc.compose(oracle_for_marked(marked), inplace=True)
    return qc

def grover_multi_circuit(marked_items, iterations=1, shots=None):
    n = len(marked_items[0])
    qc = QuantumCircuit(n, n if shots is not None else 0)
    qc.h(range(n))
    oracle = multi_marked_oracle(marked_items)
    diff = diffuser(n)
    for _ in range(iterations):
        qc.compose(oracle, inplace=True)
        qc.compose(diff, inplace=True)
    if shots is not None:
        qc.measure(range(n), range(n))
    return qc

marked_items = ["101", "110"]
qc9 = grover_multi_circuit(marked_items, iterations=1, shots=4096)
counts9 = simulator.run(transpile(qc9, simulator), shots=4096).result().get_counts()
success9 = sum(counts9.get(item, 0) for item in marked_items) / 4096
theory9 = np.sin(3 * np.arcsin(np.sqrt(2/8)))**2
print(qc9.draw(output="text"))
print("Counts:", counts9)
print(f"Probability of either marked item: measured={success9:.4f}, theory={theory9:.4f}")
print("For M=2 marked items among N=8 states, P(r)=sin^2((2r+1)arcsin(sqrt(M/N))). One iteration gives 0.5, not certainty.")
```

```
q_0: ┤ H ├──■──┤ X ├──■──┤ X ├┤ H ├┤ X ├──■──┤ X ├┤ H ├──M──»
     ├───┤┌─┴─┐└───┘┌─┴─┐└───┘└───┘└───┘┌─┴─┐└───┘└───┘  │  »
q_1: ┤ H ├┤ X ├──■──┤ X ├──■──┤ H ├┤ X ├──■──┤ X ├┤ H ├──┼──»
     ├───┤├───┤┌─┴─┐└───┘┌─┴─┐└───┘└───┘┌─┴─┐└───┘└───┘  │  »
q_2: ┤ H ├┤ H ├┤ X ├──■──┤ X ├──■──┤ H ├┤ X ├──■──┤ X ├──┼──»
     └───┘└───┘└───┘┌─┴─┐└───┘┌─┴─┐└───┘└───┘┌─┴─┐└───┘  │  »
c: 3/═══════════════╡ X ╞═════╡ X ╞═══════════╡ X ╞═══════╩══»
                    └───┘     └───┘           └───┘          0 »

«
«q_0: ─────
«
«q_1: ┤M├──
«
«q_2: ─┤M├─
«      ╚╡╞═
«c: 3/═══╩═
«        1 2
Counts: {'110': 2076, '101': 2020}
Probability of either marked item: measured=1.0000, theory=1.0000
For M=2 marked items among N=8 states, P(r)=sin^2((2r+1)arcsin(sqrt(M/N))). One iteration gives 0.5, not certainty.
```

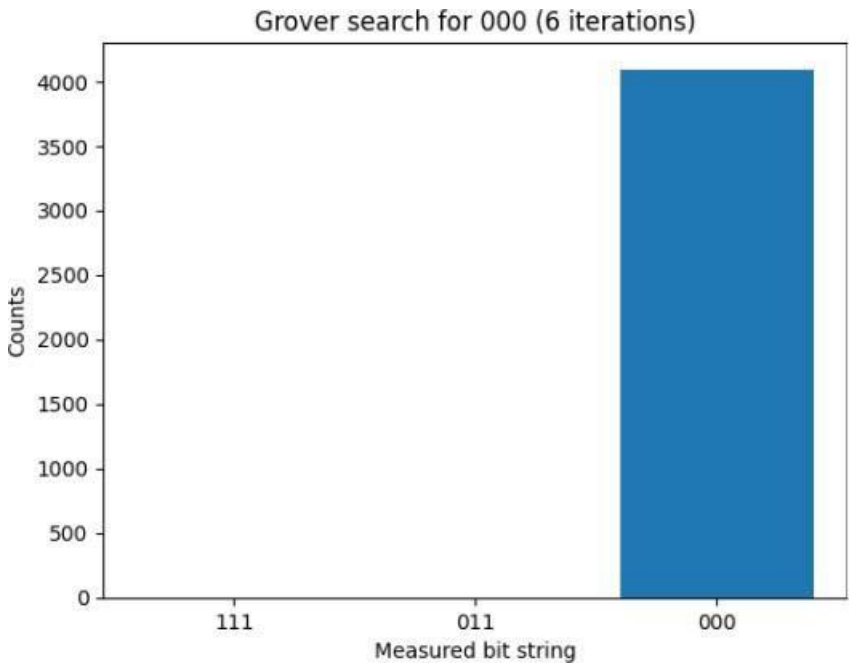
Program 10: Take the last digit of your register number, find the remainder on division by 8 and write it as a three-bit string. Use it as the marked item in the three-qubit search and report the histogram for the best number of iterations.

OUTPUT:

```
# Register number recorded for this worksheet ends in 0.
last_digit = 0
remainder = last_digit % 8
marked = format(remainder, "03b")
best_iterations = int(np.argmax([np.sin((2*r + 1) * np.arcsin(np.sqrt(1/8)))**2 for r in range(7)]))
print(f"Last digit: {last_digit}")
print(f"Remainder after division by 8: {remainder}")
print(f"Three-bit marked item: {marked}")
print(f"Best number of iterations: {best_iterations}")

qc10 = grover_circuit(marked, iterations=best_iterations, shots=4096)
counts10 = simulator.run(transpile(qc10, simulator), shots=4096).result().get_counts()
print("Counts:", counts10)
plt.bar(counts10.keys(), counts10.values())
plt.xlabel("Measured bit string")
plt.ylabel("Counts")
plt.title(f"Grover search for {marked} ({best_iterations} iterations)")
plt.show()
```

```
Last digit: 0
Remainder after division by 8: 0
Three-bit marked item: 000
Best number of iterations: 6
Counts: {'111': 1, '011': 1, '000': 4094}
```



VIVA Q&A:

1. What problem does Grover's algorithm solve?

Grover's algorithm searches an unstructured database or list to find a marked item.

2. How many steps does a classical search need for N items, and how many does Grover's algorithm need?

- **Classical search:** (O(N)) steps on average.
- **Grover's algorithm:** O(√N) oracle queries.

3. What does the oracle do to the marked item?

The oracle changes the sign of the marked state's amplitude by multiplying it by -1.

4. Why do the measurement probabilities not change immediately after the oracle?

The oracle changes only the sign of the amplitude. Since probability is the squared magnitude of the amplitude, the probabilities remain unchanged.

5. What does the diffuser do? What is meant by inversion about the mean?

The diffuser amplifies the amplitude of the marked state and reduces the amplitudes of unmarked states. Inversion about the mean means reflecting each amplitude around the average amplitude.

6. What is the best number of iterations for N = 4 and for N = 8?

For one marked item:

- **N = 4:** 1 iteration.
- **N = 8:** 2 iterations give the highest success probability among the nearby integer choices.

7. What happens if the oracle and the diffuser are repeated too many times?

The success probability decreases because the quantum state rotates past the marked state. This is called overshooting.

8. Is the speed-up of Grover's algorithm quadratic or exponential?

The speed-up is quadratic, not exponential.

9. How does the number of marked items change the number of iterations needed?

As the number of marked items increases, fewer iterations are generally needed. The approximate optimal number is:

[

$r \approx (\pi/4)\sqrt{(N/M)}$

]

where (N) is the total number of items and (M) is the number of marked items.

10. What is the effect of Grover's algorithm on symmetric-key cryptography?

Grover's algorithm can reduce the effective brute-force security of a (k)-bit key from approximately (2^k) classical operations to O(2^(k/2)) quantum oracle queries. Larger key sizes can help compensate for this effect.

RESULT

Grover's search algorithm was implemented to locate marked items in two-qubit and three-qubit search spaces. The oracle marked the target by changing its phase, and the diffuser amplified its probability of being measured. The experiment demonstrated the quadratic speed-up of Grover's algorithm and showed that the number of iterations must be chosen carefully to avoid overshooting.

EVALUATION (for faculty use):

Evaluation method	Prep	Program	Output	Inference	Viva	Timely	Total
Maximum marks	2	5	3	2	2	1	15
Marks awarded							

Faculty signature with date: